Machine Learning

CSE 343/543

Bagging, Random Forests, Boosting & Gradient Boosting Trees

Decision Tree for Regression

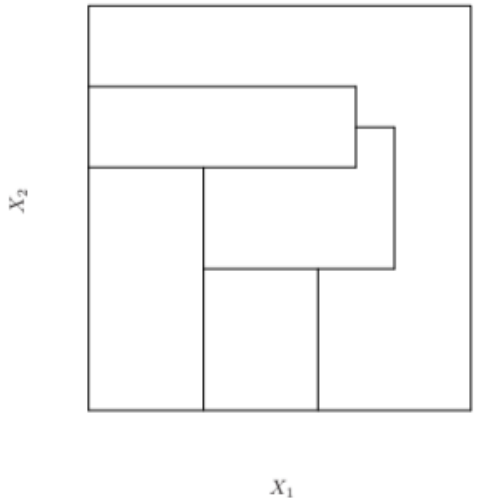
- Take a top-down, greedy, recursive splitting approach
  - At each node of the tree, pick j, s

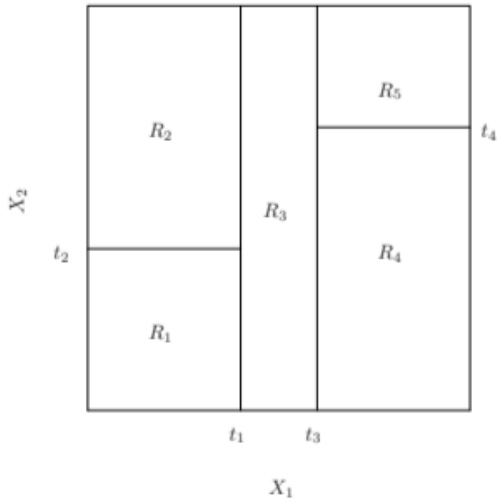
$$R_1(j,s) = \{X \mid X_j < s\} \text{ and } R_2(j,s) = \{X \mid X_j \geq s\}$$

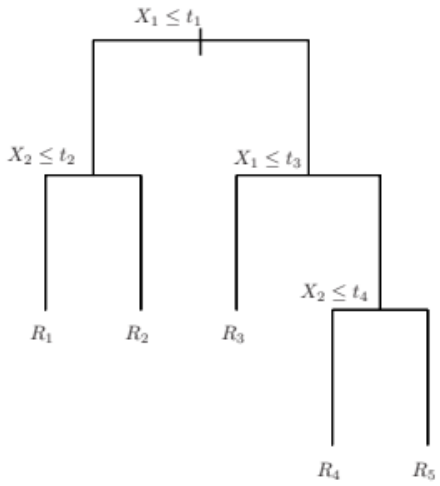
  - that minimizes

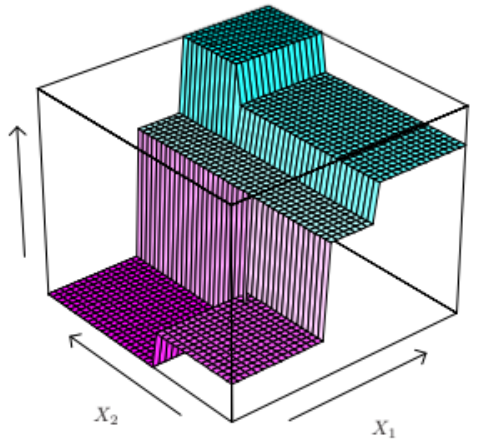
$$\sum_{i:x^{(i)} \in R_1(j,s)} \left(y^{(i)} - \hat{y}_{R_1}\right)^2 + \sum_{i:x^{(i)} \in R_2(j,s)} \left(y^{(i)} - \hat{y}_{R_2}\right)^2$$

where $\hat{y}_{R_k}$ is the ***mean response*** for the kth region R$_k$



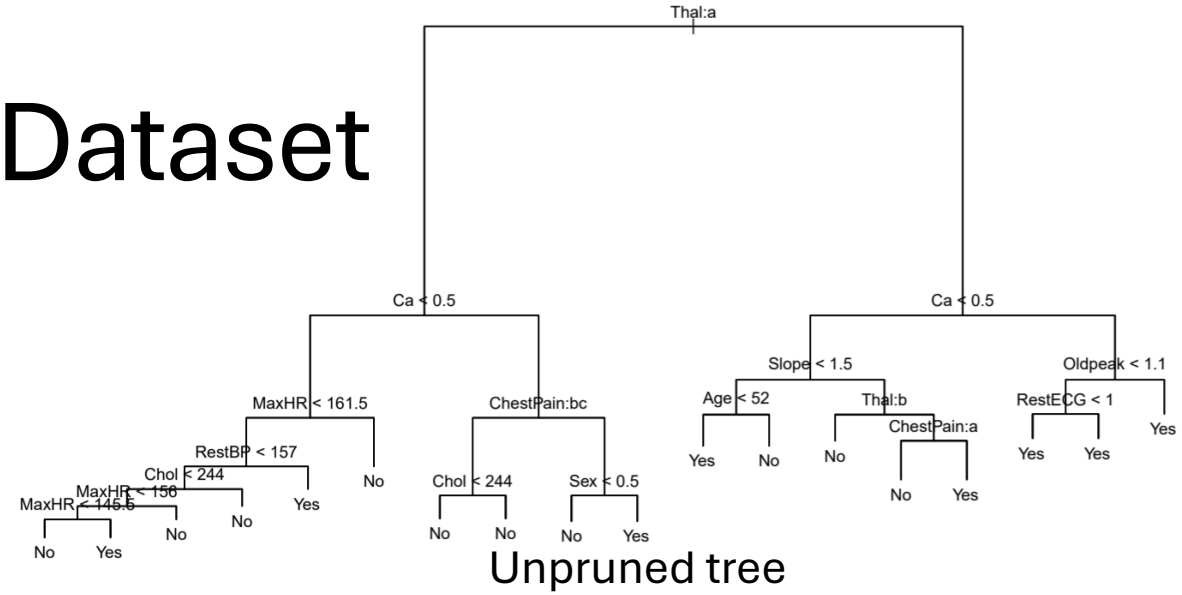




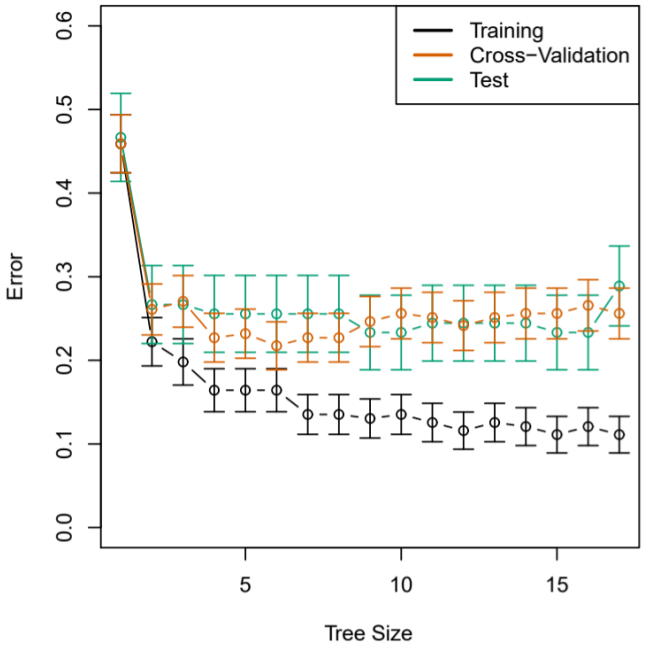


Example: Heart Disease Dataset

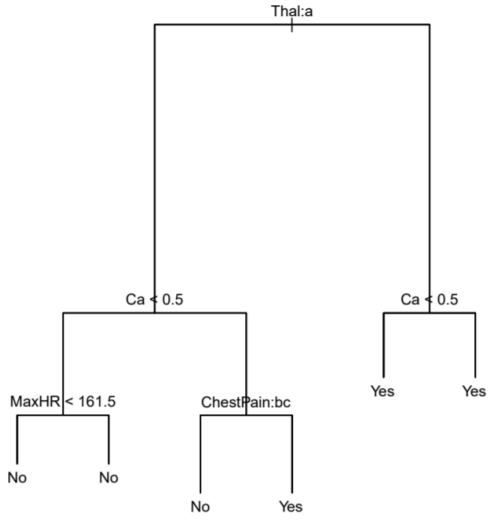
- Binary outcome (Yes/No): heart disease (HD) for 303 patients who experienced chest pain.
  - Yes / No indicates the presence / absence of heart disease (HD) based on an angiographic test.
- 13 predictors / features: Age, Sex, Chol (a cholesterol measurement), and other heart and lung function measurements.
- Cross-validation yields a tree with six terminal nodes.



Unpruned tree



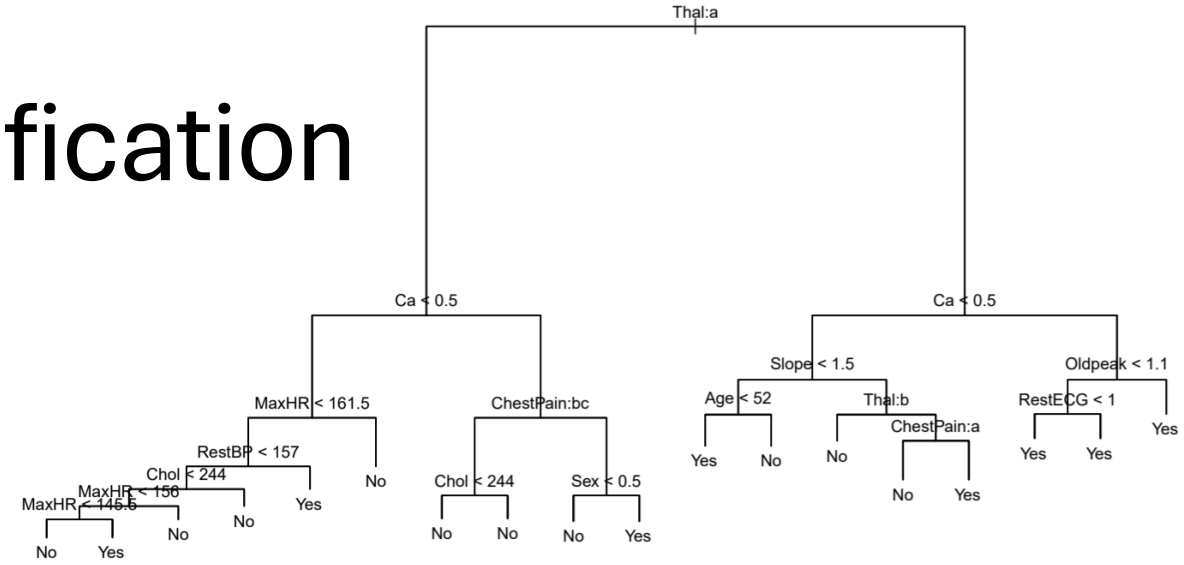
Train/val plots

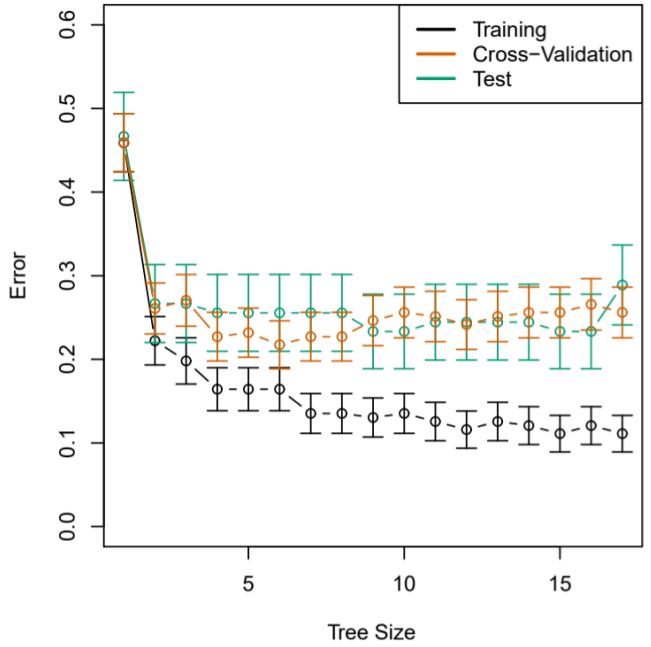


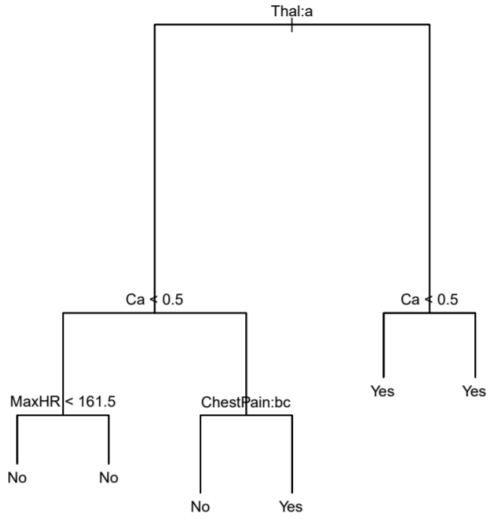
Pruned tree

Decision Trees for Classification

- For decision variable selection at train time:
  - Drop in Entropy at next node
    - $H(Y) = -\sum_{k=1}^{K} P\,(Y = y_k) \log P\,(Y = y_k)$
  - Drop in Gini Index at next node
    - $G(Y) = \sum_{k=1}^{K} P\,(Y = y_k)\big(1 - P(Y = y_k)\big)$
- At inference time
  - Majority voting for classes
  - Other ways of fusing predictions







Advantages and Disadvantages of Trees

- Trees are very easy to explain, possibly more than linear regression!
- Trees are easier to visualize graphically and are easily interpreted even by a non-expert.
- Trees can readily handle both discrete and continuous features without the need to create dummy variables.
- Decision trees generally do not achieve the same level of predictive accuracy as some of the other approaches.
  - When we use “many” trees, this changes! Enter: “Ensemble methods”

Say we have $X = \{x^1, x^2, x^3, \ldots, x^N\}$ drawn from some distribution $P(x)$

Let $Y = \frac{1}{N}\sum_{i=1}^{N} x^i$ If $E[X] = \mu$ & $Var(X) = \sigma^2$, then

$$E[Y] = E\left[\frac{1}{N}\sum_{i=1}^{N} x^i\right] = \frac{1}{N}\sum_{i=1}^{N} E[x^i] = \frac{1}{N}\cdot\sum_{i=1}^{N} E[x] = \frac{1}{N}\sum_{i=1}^{N}\mu = \frac{N\mu}{N} = \mu$$

$$Var[Y] = Var\left[\frac{1}{N}\sum_{i=1}^{N} x^i\right] = \frac{1}{N^2} Var\left[\sum_{i=1}^{N} x^i\right]$$

Since x^i, $i=1,\ldots,N$ ar i.i.d., the variance of the sum of x^i will lead to the sum of the variance; i.e., $Var[Y] = \frac{1}{N^2}\sum_{i=1}^{N} Var(x^i) = \frac{1}{N^2}\sum_{i=1}^{N}\sigma^2 = \frac{N\sigma^2}{N^2}$

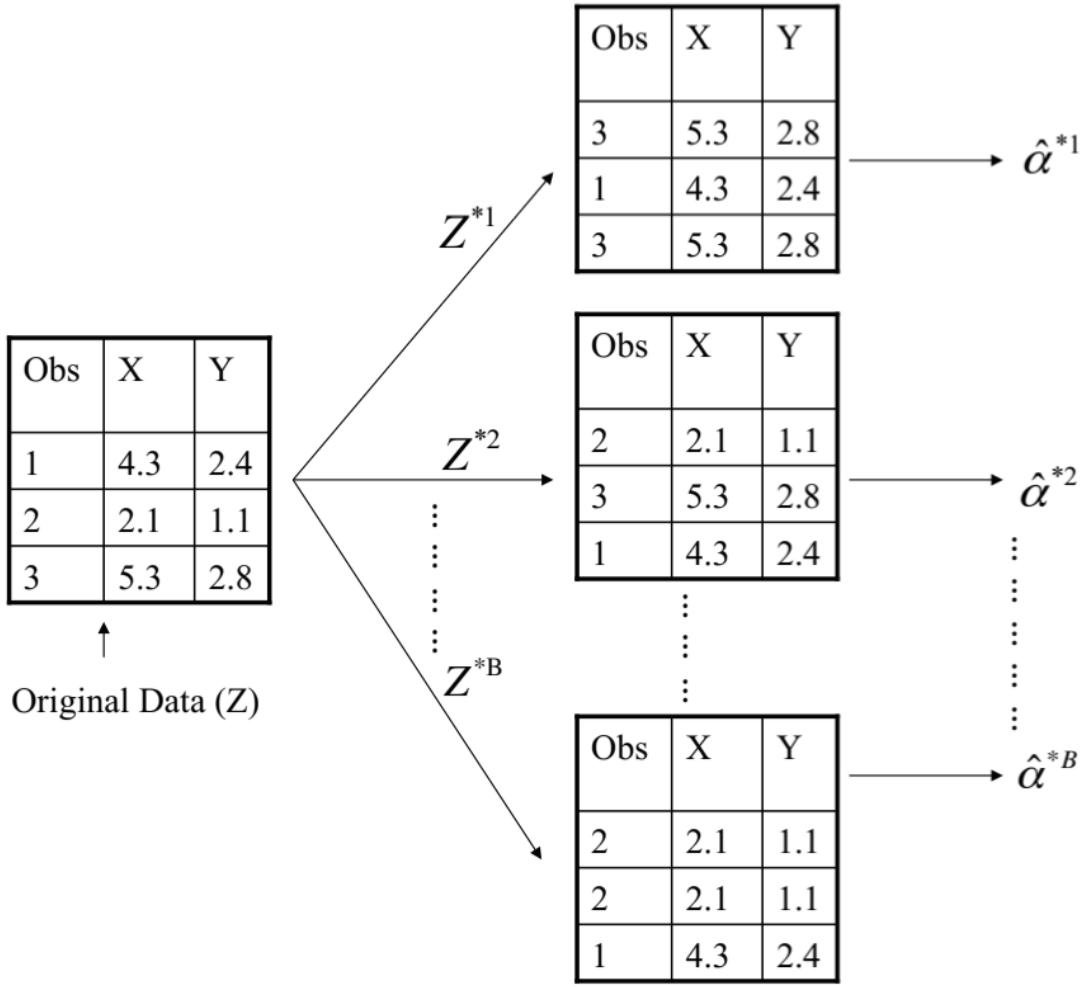
$$\therefore \boxed{Var\left[\frac{1}{N}\sum_{i=1}^{N} x^i\right] = \frac{\sigma^2}{N} \quad \& \quad E\left[\frac{1}{N}\sum_{i=1}^{N} x^i\right] = E[x] = \mu}$$

Averaging of a random variable, when independently drawn, can lead to a reduction in variance.

This is applicable to model predictions as well and inspires an entire family of methods — Bootstrapped Aggregation or Bagging.

Bootstrapping

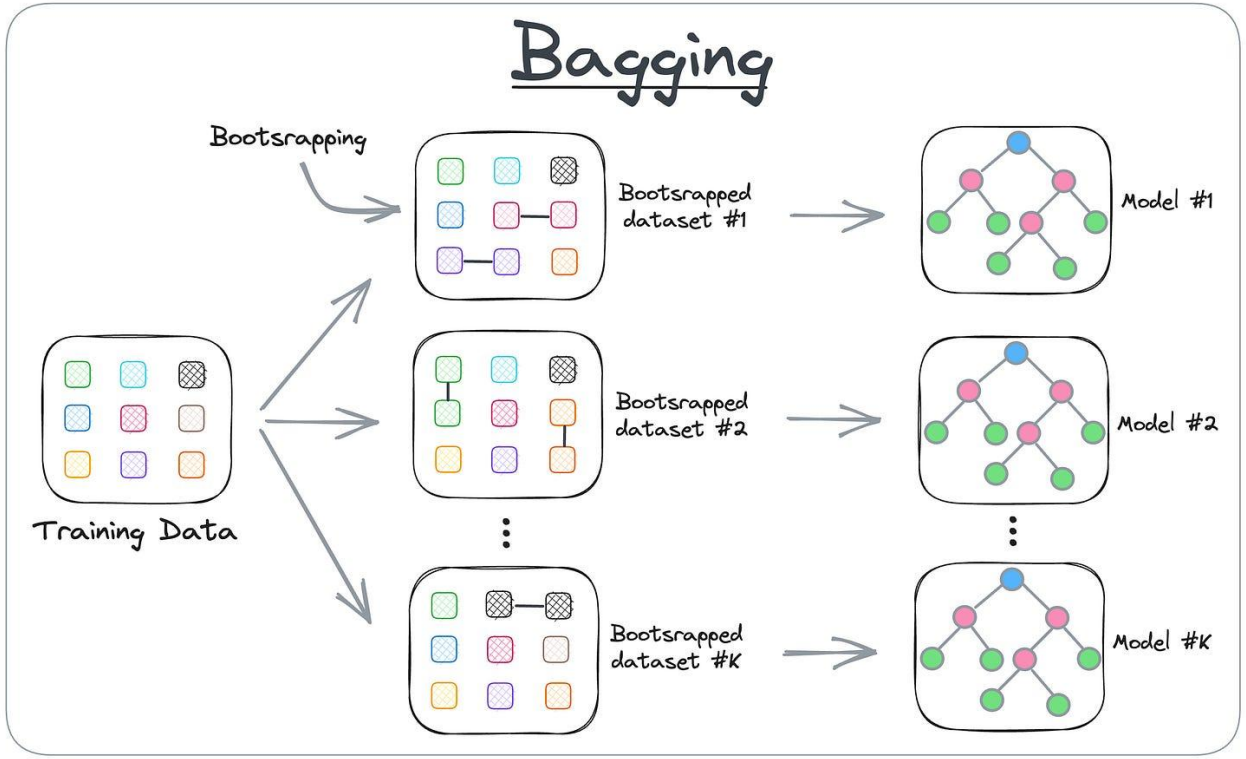
- Given a dataset D of N data points
- Sample with replacement
  - Results in B datasets containing some repeated data points.
  - Estimate the statistic for each of the B datasets
  - Average the estimate over the B datasets
  - The resulting *Bootstrapped estimate* yields a lower variance
  - Applicable to ML models for reducing variance, especially Decision Trees



Bootstrap Aggregation

- Generate B bootstrapped datasets
- Train B unpruned decision trees, one on each dataset
- At inference time, generate predictions, and average them

$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$$



Probability of a sample being *Out of Bag* (OOB)

Let $X = \{x^j\}_{j=1}^{N}$ be the original dataset.

Let the bootstrapped dataset by $X_b = \{x_b^{(i)}\}_{i=1}^{N}$

which is created by sampling with replacement from X, the original dataset.

The probability of $x^j \in X$ being the $i=1^{st}$ sample of X_b is $P(x_b^{(1)} = x^j) = \frac{1}{N}$

$\therefore$ The prob. of $x^j \in X$ NOT being the

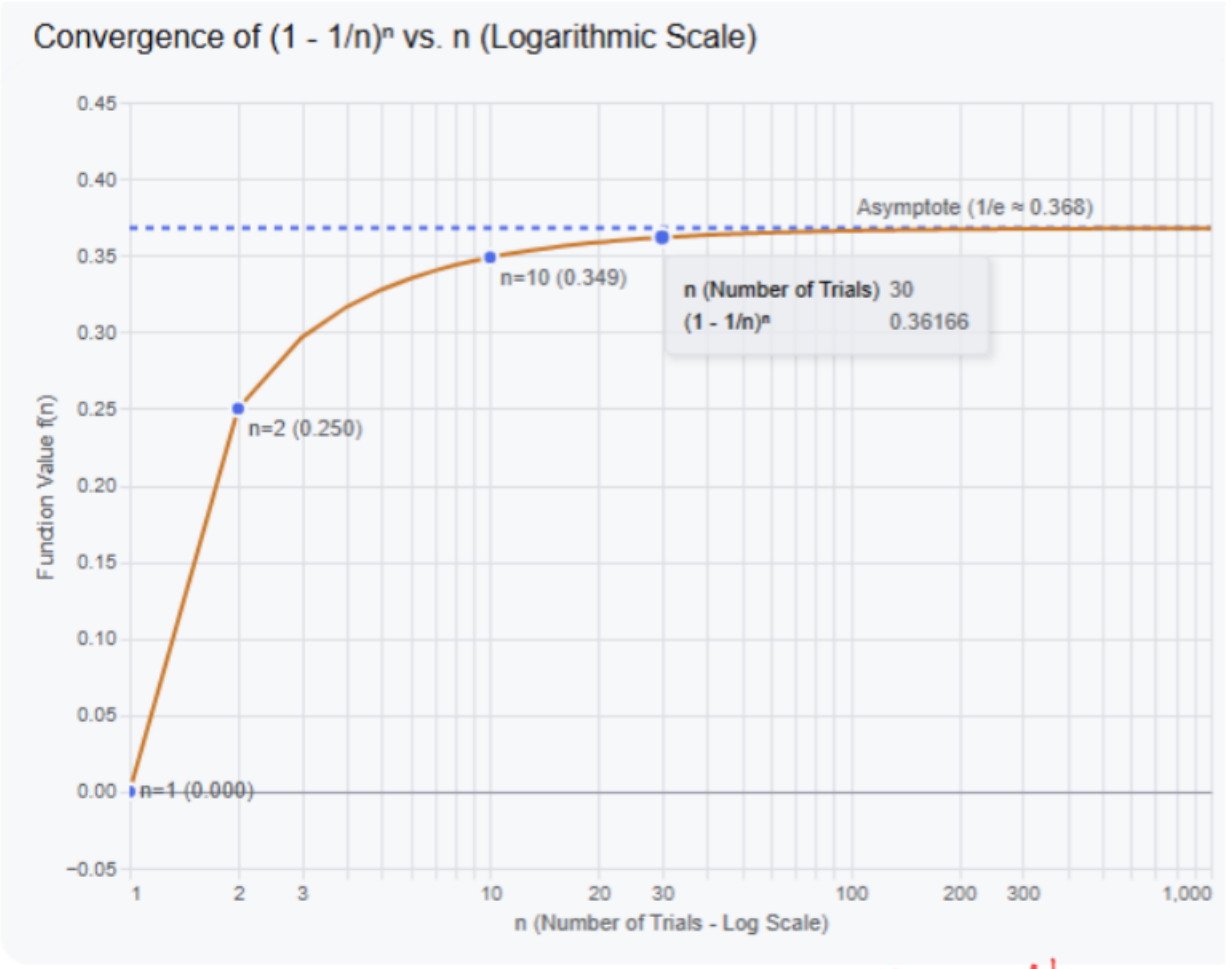
$\rightarrow$ $i=1^{st}$ sample of X_b is $P(x_b^{(1)} \neq x^j) = 1 - \frac{1}{N}$

$\rightarrow$ $i=2^{nd}$ sample of X_b is $P(x_b^{(2)} \neq x^j) = 1 - \frac{1}{N}$

$\Rightarrow$ The prob. $x^j \in X$ NOT being at any $i=1,\dots,N$ in X_b is $P(x^j \notin X_b) = \left(1 - \frac{1}{N}\right)^N$

$\simeq 0.36$

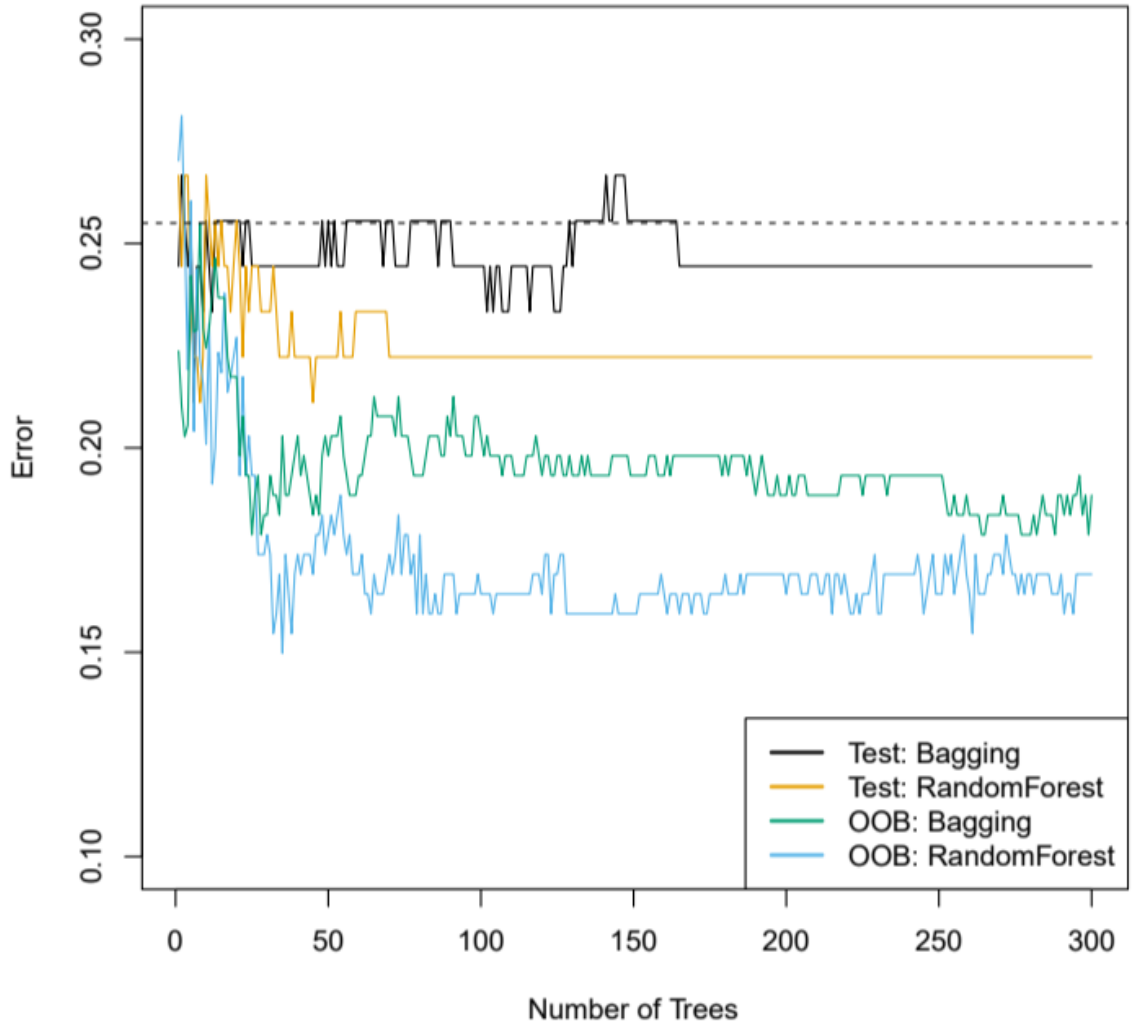
$x^j \in X_b$ is said to be Out of Bag for bag b



for sufficiently large N $\because \lim_{N\to\infty}\left(1 - \frac{1}{N}\right)^N = \frac{1}{e} \simeq 0.36$

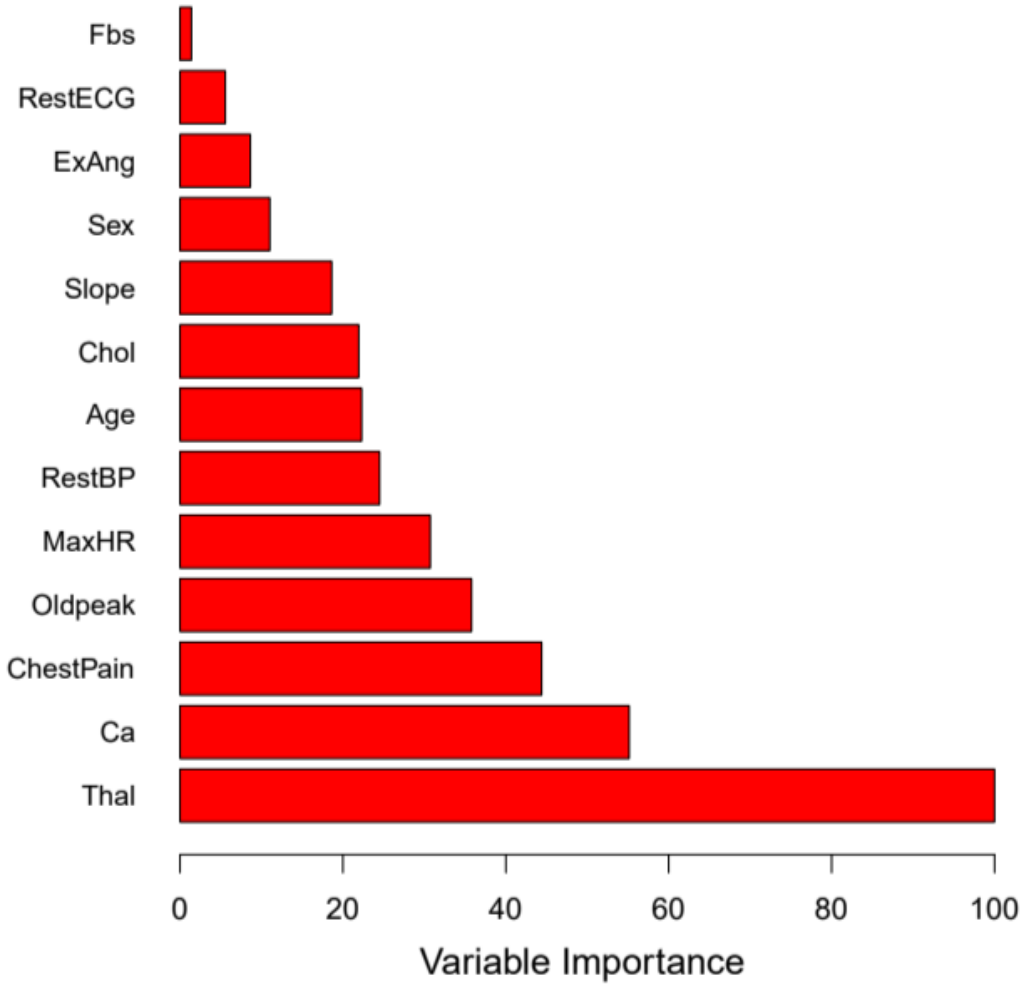
Performance Evaluation of Bagged Trees

- Bagging reduces variance, addresses overfitting
- However, at the expense of Interpretability



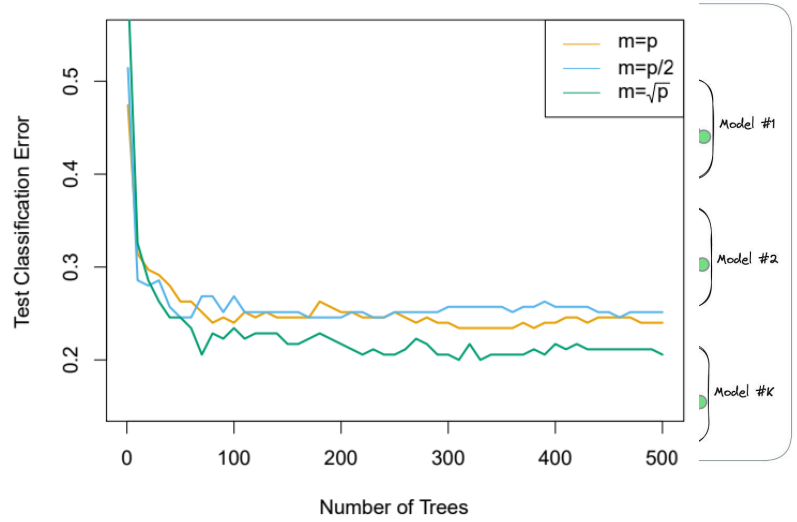
Variable Importance Measures in Bagged DTs

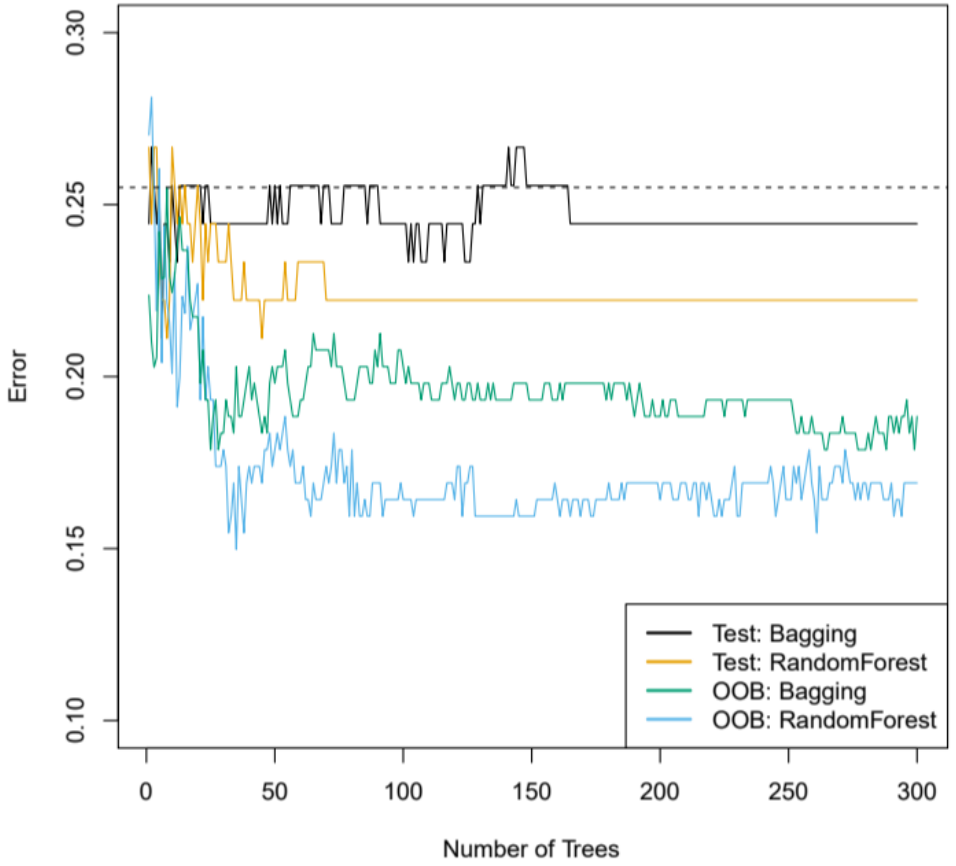
- Explainability is lost due to multiple trees and predictions being averaged / voted
- Variable / feature / predictor importance can be measured
  - Record total amount of error reduced due to a feature used for a split
  - Average this number across all B trees
  - Plot bar graph of relative feature importance



Random Forests

- Same as Bagging, but with *decorrelated* trees
- At each split, restrict decision variable to a small random subset of all variables / predictors
  - Typical to choose $\sqrt{p}$, where p is the set of all predictors / variables
- Results in decorrelated trees
  - Recall that averaging of independent samples reduces variance
  - Bagging with heavily correlated trees will not reduce variance sufficiently





Boosting Decision Trees

- Bagging uses resampling to *independently* & *parallelly* train multiple trees
- Boosting trains multiple trees *sequentially*: each tree is grown using information from previously grown trees.
  - No bootstrap sampling, instead each tree is fit on a modified version of the original data set.
  - The modification is based on the prediction errors of the previous tree
    - The predictors / input variables are the same, but the targets are the “residuals” between the original target and the previous models’ predictions.
- Boosting benefits the most from ‘weak learners’
  - e.g., decision stumps

Algorithm 8.2 *Boosting for Regression Trees*

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.

2. For $b = 1, 2, \ldots, B$, repeat:

   (a) Fit a tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to the training data (X, r).

   (b) Update $\hat{f}$ by adding in a shrunken version of the new tree:

   $$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x).$$

   (c) Update the residuals,

   $$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

3. Output the boosted model,

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x).$$

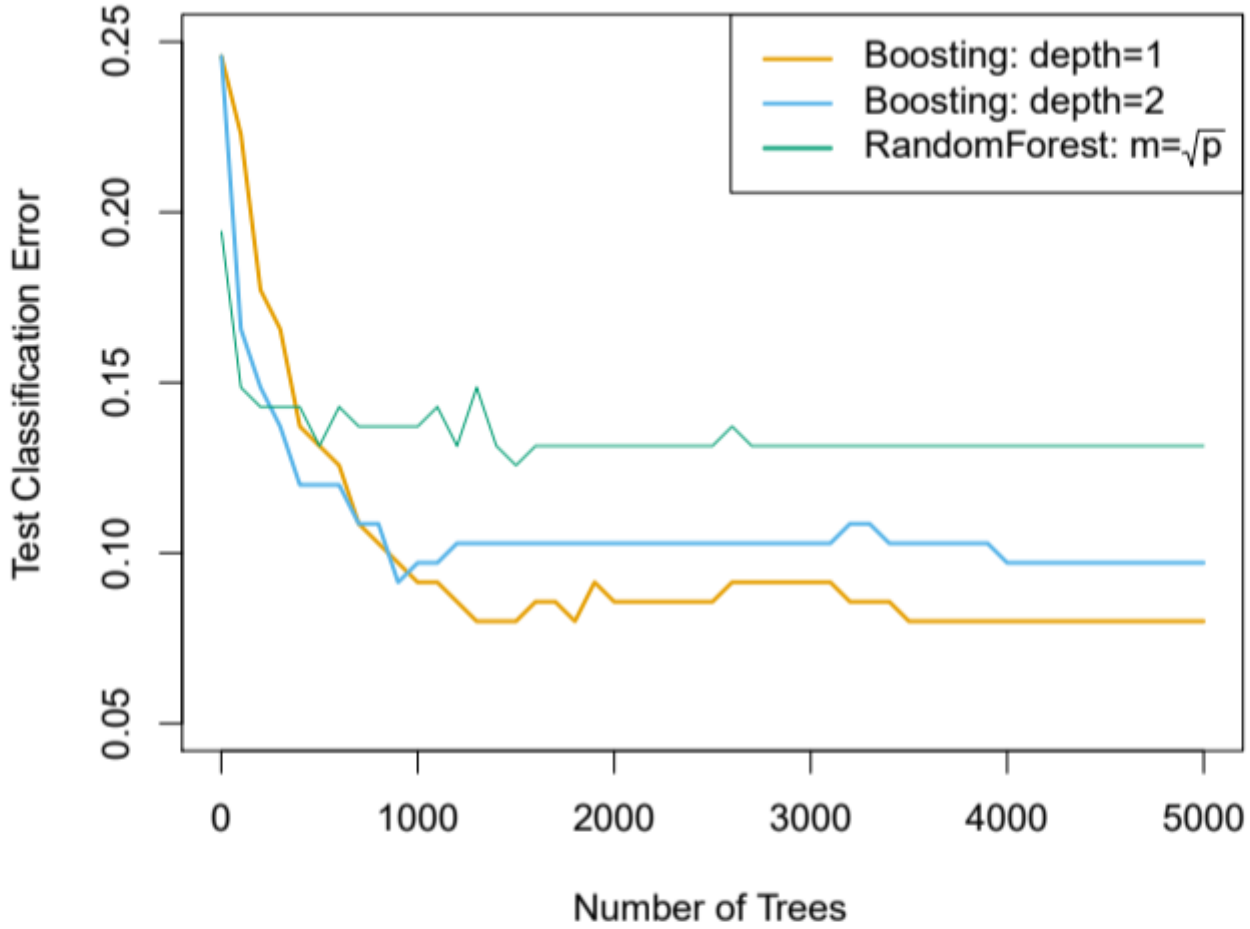


FIGURE 8.11. *Results from performing boosting and random forests on the 15-class gene expression data set in order to predict* cancer *versus* normal. *The test error is displayed as a function of the number of trees. For the two boosted models,* $\lambda = 0.01$. *Depth-1 trees slightly outperform depth-2 trees, and both outperform the random forest, although the standard errors are around 0.02, making none of these differences significant. The test error rate for a single tree is 24 %.*

Algorithm 10.1 *AdaBoost.M1.*

1. Initialize the observation weights $w_i = 1/N,\ i = 1, 2, \ldots, N$.
2. For $m = 1$ to M:
   (a) Fit a classifier $G_m(x)$ to the training data using weights w_i.
   (b) Compute
   $$\text{err}_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}.$$
   (c) Compute $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$.
   (d) Set $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))],\ i = 1, 2, \ldots, N$.
3. Output $G(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$.

$$Y = \begin{cases} 1 & \text{if } \sum_{j=1}^{10} X_j^2 > \chi_{10}^2(0.5), \\ -1 & \text{otherwise.} \end{cases}$$

Here $\chi_{10}^2(0.5) = 9.34$ is the median of a chi-squared random variable with 10 degrees of freedom (sum of squares of 10 standard Gaussians). There are 2000 training cases, with approximately 1000 cases in each class, and 10,000 test observations. Here the weak classifier is just a "stump": a two terminal node classification tree.

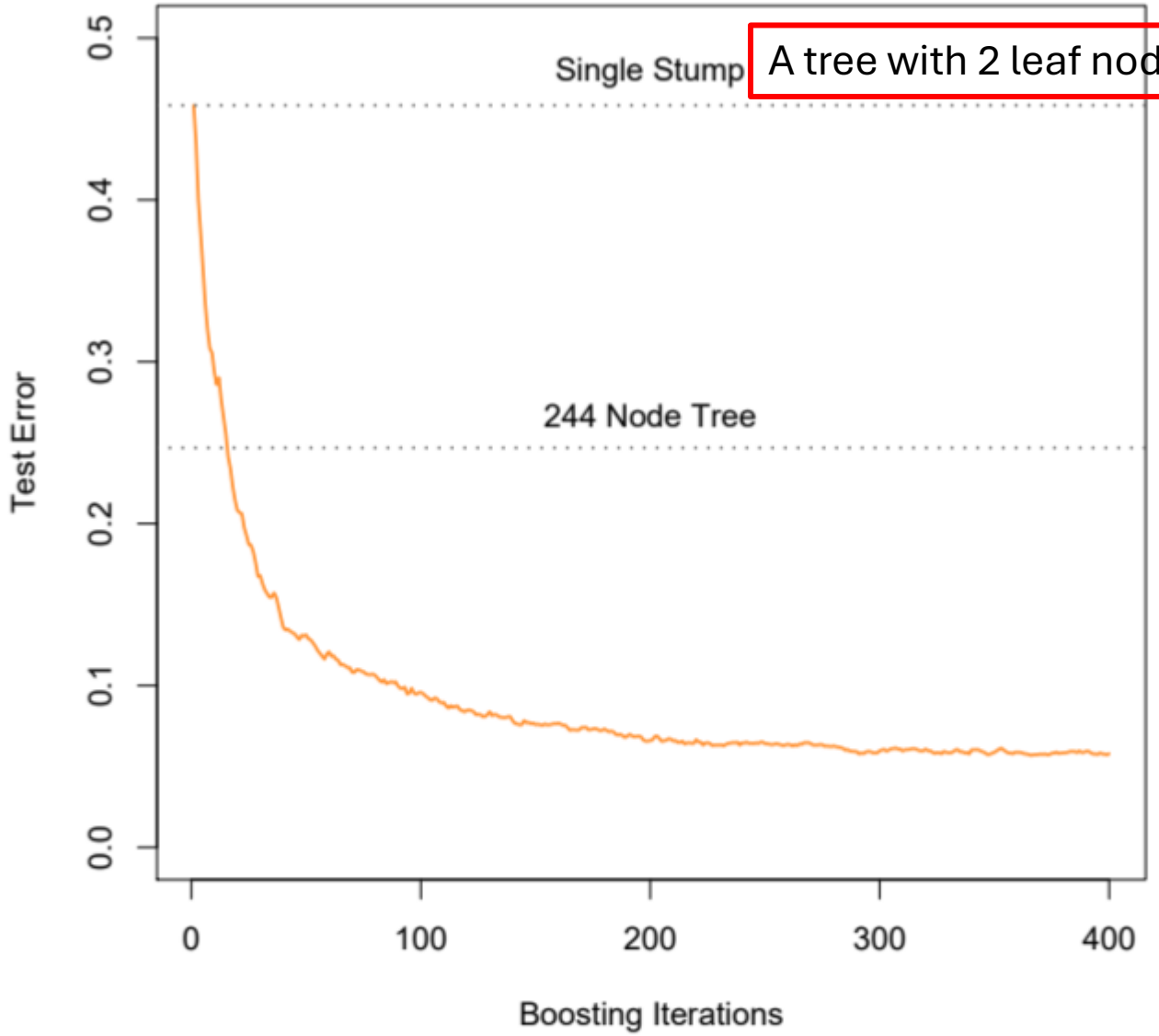


FIGURE 10.2. *Simulated data (10.2): test error rate for boosting with stumps, as a function of the number of iterations. Also shown are the test error rate for a single stump, and a 244-node classification tree.*

An Abstraction for Boosting Methods

Algorithm 10.2 *Forward Stagewise Additive Modeling.*

1. Initialize $f_0(x) = 0$.
2. For $m = 1$ to M:
   (a) Compute

$$(\beta_m, \gamma_m) = \arg\min_{\beta,\gamma} \sum_{i=1}^{N} L(y_i, f_{m-1}(x_i) + \beta b(x_i; \gamma)).$$

   (b) Set $f_m(x) = f_{m-1}(x) + \beta_m b(x; \gamma_m)$.

Boosting Trees

- Tree:

$$T(x;\Theta) = \sum_{j=1}^{J} \gamma_j I(x \in R_j),$$

$$\Theta = \{R_j, \gamma_j\}_1^J$$

Partitioning the feature space into J regions and assigning γ_j as the predicted value for the jth region.

- Trained via an appropriate loss function

$$\hat{\Theta} = \arg\min_{\Theta} \sum_{j=1}^{J} \sum_{x_i \in R_j} L(y_i, \gamma_j).$$

- Combinatorial problem ➔ Greedy solution with recursive splitting
  - Step 1: Finding R_j Step 2: Finding γ_j
  - and iterate
- Boosted Trees are a sum of such trees

$$f_M(x) = \sum_{m=1}^{M} T(x;\Theta_m)$$

Gradient Boosting Trees

$$\hat{\Theta}_m = \arg\min_{\Theta_m} \sum_{i=1}^{N} L\left(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m)\right)$$

TABLE 10.2. *Gradients for commonly used loss functions.*

Setting	Loss Function	$-\partial L(y_i, f(x_i))/\partial f(x_i)$
Regression	$\frac{1}{2}[y_i - f(x_i)]^2$	$y_i - f(x_i)$
Regression	$\lvert y_i - f(x_i)\rvert$	$\text{sign}[y_i - f(x_i)]$
Regression	Huber	$y_i - f(x_i)$ for $\lvert y_i - f(x_i)\rvert \le \delta_m$ $\delta_m \text{sign}[y_i - f(x_i)]$ for $\lvert y_i - f(x_i)\rvert > \delta_m$ where $\delta_m = \alpha$th-quantile$\{\lvert y_i - f(x_i)\rvert\}$
Classification	Deviance	kth component: $I(y_i = \mathcal{G}_k) - p_k(x_i)$

Algorithm 10.3 *Gradient Tree Boosting Algorithm.*

1. Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.

2. For $m = 1$ to M:

   (a) For $i = 1, 2, \ldots, N$ compute

   $$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}.$$

   (b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm}, $j = 1, 2, \ldots, J_m$.

   (c) For $j = 1, 2, \ldots, J_m$ compute

   $$\gamma_{jm} = \arg\min_\gamma \sum_{x_i \in R_{jm}} L\left(y_i, f_{m-1}(x_i) + \gamma\right).$$

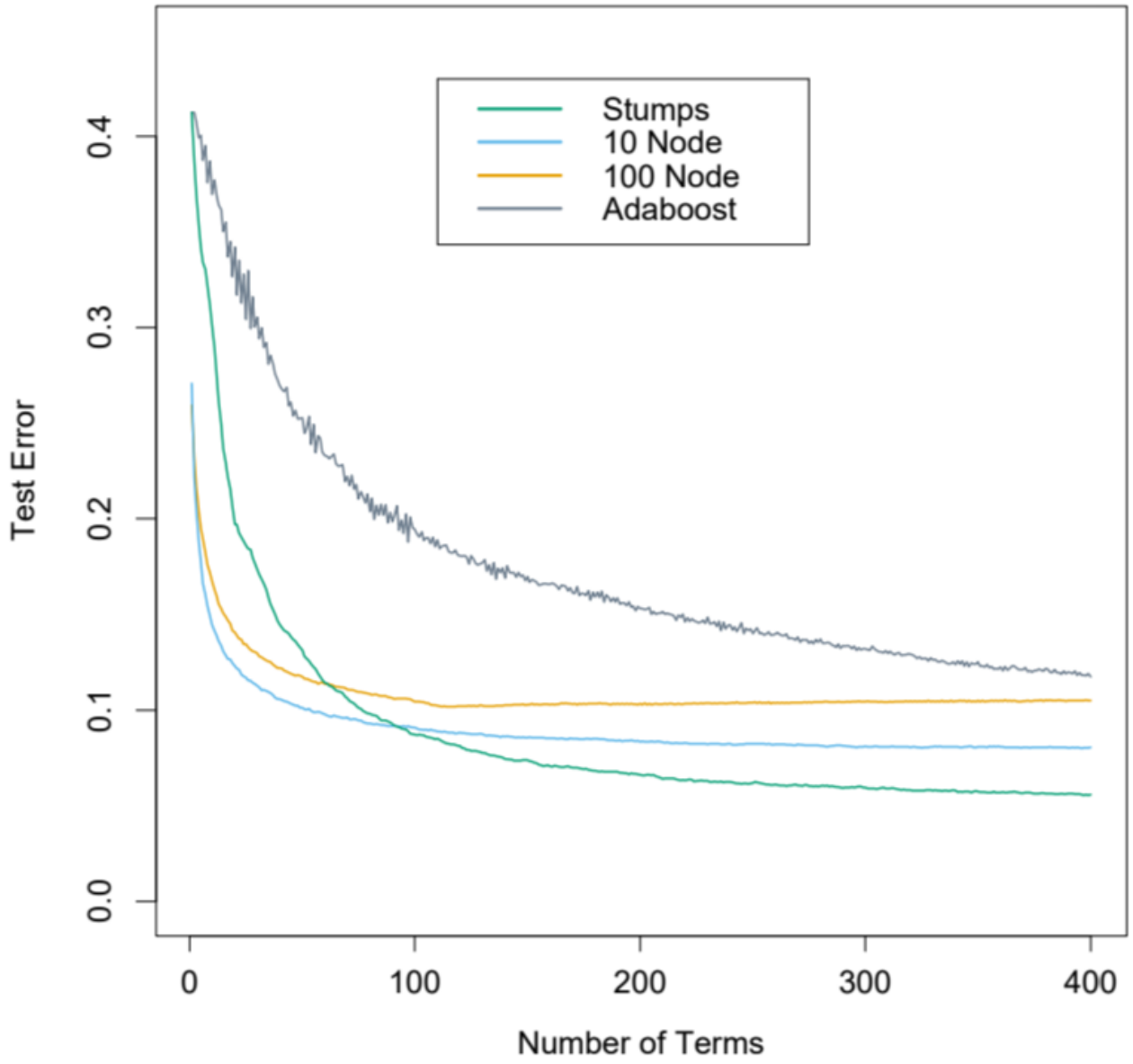
   (d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.

3. Output $\hat{f}(x) = f_M(x)$.

Tree Size for Boosting

- Boosting results in highest improvements when we use weak learners

- The plot shows stumps (2-leaf node trees) eventually perform better for larger ensembles



Summary

- Bagging or Bootstrap Aggregating is an Ensemble Method that samples multiple datasets to train different models
  - At inference the predictions from all models are used for averaging / voting
  - Reduces prediction variance, but also interpretability
  - Applied to Decision Trees, when combined with feature decorrelation, gives Random Forests, a powerful ML model
- Boosting is another Ensemble Method that uses the same dataset, but trains a cascade of *weak learners / models* to improve prediction accuracy

Next Class

- Large Margin Classifiers and Support Vector Machines
- Convex Functions and Convex Optimization